PROPOSAL TUGAS AKHIR - EE234799

PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

PROPOSAL TUGAS AKHIR - EE234799

PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]

FINAL PROJECT PROPOSAL - EE234799

PERFORMANCE COMPARISON OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED HEADPHONE ACTIVE NOISE CANCELLATION (ANC) SYSTEM

KEINDRA DANTE HIDAYAT

NRP 5022231236

Advisor

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : KEINDRA DANTE HIDAYAT
Judul Tugas Akhir : PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA
Pembimbing : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) digunakan untuk meredam kebisingan frekuensi rendah yang tidak dapat diatasi secara efektif oleh peredam pasif. Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar pada sistem ANC, tetapi *step-size* yang tetap membuat kinerjanya bergantung pada *tuning* manual dan rentan terhadap variasi daya sinyal kebisingan *broadband*. *Filtered-X Normalized Least Mean Square* (FXNLMS) mengatasi kelemahan tersebut dengan menormalisasi *step-size* terhadap daya sinyal referensi terfilter, tetapi menambah estimasi daya dan operasi pembagian pada setiap iterasi. Penelitian terdahulu umumnya hanya menyoroti kinerja peredaman atau efisiensi perangkat keras secara terpisah, sehingga belum diketahui apakah keunggulan FXNLMS sepadan dengan biaya komputasi tambahannya. Penelitian ini membandingkan FXLMS dan FXNLMS pada sistem ANC *headphone feedforward single-channel* (SISO) melalui tiga tahap, yaitu simulasi kinerja peredaman menggunakan *white noise*, *pink noise*, dan rekaman bising lingkungan nyata; analisis pengaruh normalisasi *step-size* terhadap kecepatan konvergensi dan *steady-state error*; serta implementasi *fixed-point* pada FPGA Artix-7 XC7A35T. Kinerja dievaluasi melalui peredaman kebisingan (SNR), kecepatan konvergensi, dan *steady-state error*, sedangkan efisiensi komputasi dievaluasi melalui kompleksitas operasi per-iterasi serta utilisasi LUT, *flip-flop*, memori, dan DSP *slice*. Hasil penelitian diharapkan memberikan dasar kuantitatif dalam memilih algoritma ANC untuk perangkat tersemat berkapasitas terbatas.

Kata Kunci: *Active Noise Cancellation*, FXLMS, FXNLMS, FPGA, Efisiensi Komputasi

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : KEINDRA DANTE HIDAYAT
Title : *PERFORMANCE COMPARISON OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED HEADPHONE ACTIVE NOISE CANCELLATION (ANC) SYSTEM*
Advisors : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) is used to attenuate low-frequency noise that passive attenuation cannot suppress. The Filtered-X Least Mean Square (FXLMS) algorithm is the standard in ANC systems, but its fixed step-size makes its performance dependent on manual tuning and sensitive to power variations in broadband noise. The Filtered-X Normalized Least Mean Square (FXNLMS) algorithm addresses this weakness by normalizing the step-size to the power of the filtered reference signal, but adds a power estimate and a division per iteration. Previous studies generally examined either noise reduction performance or hardware efficiency separately, so it remains unclear whether FXNLMS justifies its additional computational cost. This research compares FXLMS and FXNLMS in a feedforward single-channel (SISO) headphone ANC system in three stages: noise reduction simulation using white noise, pink noise, and real-world ambient noise recordings; analysis of how step-size normalization affects convergence speed and steady-state error; and fixed-point implementation on an Artix-7 XC7A35T FPGA. Performance is evaluated through noise reduction (SNR), convergence speed, and steady-state error, while computational efficiency is evaluated through per-iteration operation complexity and the utilization of LUTs, flip-flops, memory, and DSP slices. The results are expected to give a quantitative basis for selecting ANC algorithms for resource-constrained embedded devices.

Keywords: *Active Noise Cancellation*, *FXLMS*, *FXNLMS*, *FPGA*, *Computational Efficiency*

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK i

ABSTRACT iii

DAFTAR ISI v

DAFTAR GAMBAR vii

DAFTAR TABEL ix

1 PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Rumusan Masalah 2

1.3 Batasan Masalah 2

1.4 Tujuan 2

1.5 Manfaat 3

1.5.1 Bagi Peneliti 3

1.5.2 Bagi Masyarakat Umum 3

2 TINJAUAN PUSTAKA 5

2.1 Hasil Penelitian Terdahulu 5

2.2 Dasar Teori 7

2.2.1 *Active Noise Cancellation* (ANC) 7

2.2.2 Filter Adaptif dan Algoritma LMS 8

2.2.3 Algoritma FXLMS 9

2.2.4 Algoritma FXNLMS 10

2.2.5 Representasi *Fixed-Point* 12

2.2.6 *Field-Programmable Gate Array* (FPGA) 12

2.2.7 Antarmuka Audio I2S 13

3 METODOLOGI 15

3.1 Metode yang Digunakan 15

3.1.1 Metode Simulasi dan Evaluasi Kinerja Peredaman 15

3.1.2 Metode Analisis Pengaruh Normalisasi *Step-Size* 16
3.1.3 Metode Implementasi pada FPGA dan Evaluasi Efisiensi Komputasi . . 16
3.2 Bahan dan Peralatan yang Digunakan 17
3.3 Urutan Pelaksanaan Penelitian 18
3.4 Jadwal Pelaksanaan 19

DAFTAR PUSTAKA **21**

DAFTAR GAMBAR

3.1 Diagram Alir Pelaksanaan Penelitian 19

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

2.1 Hasil Penelitian Terdahulu 5
2.2 Perbandingan FXLMS dan FXNLMS 11

3.1 Rancangan Awal Algoritma dan Dugaan Awal 16
3.2 Bahan dan Peralatan yang Digunakan 17
3.3 Jadwal Pelaksanaan 20

[Halaman ini sengaja dikosongkan]

BAB I
PENDAHULUAN

1.1 Latar Belakang

Polusi suara mendegradasi kenyamanan akustik pada berbagai lingkungan, mulai dari kabin kendaraan hingga perangkat audio portabel. *Passive Noise Cancellation* (PNC) efektif meredam kebisingan frekuensi tinggi, namun performanya menurun drastis pada frekuensi rendah [1, 2]. *Active Noise Cancellation* (ANC) diterapkan untuk mengatasi keterbatasan tersebut melalui prinsip interferensi destruktif secara *real-time*.

Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar dalam ANC karena mampu mengompensasi jalur sekunder dan menjaga stabilitas konvergensi [2, 3]. Idealnya, algoritma kontrol adaptif pada sistem ANC memberikan konvergensi cepat dan stabil terhadap kebisingan *broadband* yang bersifat non-stasioner tanpa membebani sumber daya komputasi perangkat secara berlebihan, terutama pada perangkat tersemat berkapasitas terbatas seperti *headphone*. Pada praktiknya, FXLMS standar menggunakan *step-size* (μ) tetap sehingga kinerjanya bergantung pada *tuning* manual dan rentan terhadap variasi daya sinyal pada kebisingan *broadband* tersebut [4]. *Filtered-X Normalized Least Mean Square* (FXNLMS) dikembangkan untuk mengatasi kelemahan ini dengan menormalisasi *step-size* secara adaptif berdasarkan daya sinyal referensi terfilter. Pada aplikasi otomotif, pendekatan ini dilaporkan meningkatkan peredaman kebisingan hingga rata-rata 5,07 dB dengan performa terbaik di bawah 1 kHz, sekaligus mempercepat konvergensi dibandingkan FXLMS standar [1, 4].

Peningkatan performa tersebut disertai konsekuensi pada sisi komputasi. Normalisasi *step-size* pada FXNLMS menambah beban berupa estimasi daya sinyal dan operasi pembagian pada setiap iterasi, yang berdampak pada kompleksitas implementasi di perangkat tersemat berkapasitas terbatas [3, 4]. Sejauh ini, penelitian yang membandingkan FXLMS dan FXNLMS umumnya hanya menyoroti satu sisi dari persoalan tersebut: performa peredaman melalui simulasi perangkat lunak [1, 2, 3, 4], atau efisiensi sumber daya perangkat keras pada algoritma adaptif lain secara terpisah, seperti reduksi penggunaan sumber daya logika FPGA sebesar 12,5% hingga 53,3% pada implementasi *Signed-Error* LMS [5]. Belum ada kajian yang menempatkan kedua sisi ini secara berdampingan untuk kebisingan *broadband* pada sistem ANC *headphone*, sehingga belum diketahui secara empiris apakah keunggulan konvergensi FXNLMS sepadan dengan biaya komputasi tambahan yang ditimbulkannya.

Kesenjangan inilah yang mendasari tiga pertanyaan yang diajukan dalam penelitian ini. Pertama, sejauh mana FXNLMS unggul dibandingkan FXLMS standar dalam meredam kebisingan *broadband* pada sistem ANC *headphone*. Kedua, apakah normalisasi *step-size* benar-benar mempercepat konvergensi dan menurunkan *steady-state error* saat menghadapi fluktuasi sinyal *broadband*, mengingat *step-size* tetap pada FXLMS menghadirkan dilema klasik antara kecepatan dan kestabilan. Ketiga, seberapa besar tambahan beban komputasi yang harus ditanggung untuk memperoleh keunggulan tersebut, dilihat dari kompleksitas operasi per-iterasi maupun konsumsi sumber daya logika pada FPGA.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana perbandingan kinerja algoritma FXLMS dan FXNLMS dalam meredam sinyal kebisingan tipe *broadband* pada sistem ANC *headphone*?
2. Bagaimana pengaruh mekanisme normalisasi *step-size* pada algoritma FXNLMS terhadap kecepatan konvergensi dan *steady-state error* saat menangani fluktuasi sinyal *broadband*, jika dibandingkan dengan algoritma FXLMS standar yang menggunakan *step-size* tetap?
3. Bagaimana tingkat efisiensi komputasi algoritma FXNLMS dibandingkan FXLMS standar, ditinjau dari kompleksitas operasi per-iterasi (jumlah operasi perkalian, penjumlahan, dan pembagian) serta utilisasi sumber daya logika blok DSP, MAC, dan LUT pada arsitektur FPGA, mengingat FXNLMS memerlukan operasi normalisasi tambahan?

1.3 Batasan Masalah

Batasan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Topologi Sistem: Perancangan arsitektur *Active Noise Cancellation* (ANC) dibatasi pada sistem *Feedforward Single-Channel* (SISO - *Single-Input Single-Output*), yang merepresentasikan model pemrosesan akustik untuk satu sisi *earpiece* pada *headphone*.
2. Cakupan Algoritma: Algoritma filter adaptif yang diimplementasikan dan dikomparasikan secara eksklusif hanya algoritma *Filtered-X Least Mean Square* (FXLMS) standar dengan *step-size* tetap dan *Filtered-X Normalized Least Mean Square* (FXNLMS) dengan *step-size* ternormalisasi.
3. Karakteristik Sinyal Bising: Sinyal referensi atau gangguan yang dievaluasi difokuskan pada kebisingan spektrum lebar (*broadband noise*) pada pita frekuensi rendah (rentang frekuensi di bawah 1000 Hz–2000 Hz). Sinyal uji direpresentasikan menggunakan *white noise*, *pink noise*, atau rekaman bising lingkungan nyata (*real-world ambient noise*).
4. Metrik Evaluasi Kinerja dan Efisiensi: Evaluasi kinerja dibatasi pada metrik peredaman kebisingan (*noise reduction*) yang diukur dalam SNR (*Signal-to-Noise Ratio*), kecepatan konvergensi, dan *steady-state error* hasil simulasi. Evaluasi efisiensi komputasi dibatasi pada kompleksitas operasi aritmetika per-iterasi serta metrik sintesis logika pada platform *Field-Programmable Gate Array* (FPGA), yaitu utilisasi sumber daya (LUT, *Flip-Flop*, memori, dan DSP *Slices*).

1.4 Tujuan

Berdasarkan rumusan masalah, tujuan dari penelitian ini adalah sebagai berikut:

1. Menganalisis secara komparatif kemampuan algoritma FXLMS dan FXNLMS dalam mereduksi sinyal kebisingan spektrum lebar pada sistem ANC *headphone*.

2. Mengukur dan mengidentifikasi pengaruh mekanisme normalisasi *step-size* pada algoritma FXNLMS terhadap performa metrik kontrol, khususnya pada kecepatan konvergensi (*convergence rate*) dan nilai galat kondisi tunak (*steady-state error*), jika dibandingkan dengan performa FXLMS standar.

3. Menghitung dan membandingkan tingkat efisiensi komputasi antara algoritma FXLMS dan FXNLMS, mencakup kompleksitas operasi per-iterasi serta konsumsi sumber daya perangkat keras digital LUT, *Flip-Flops*, dan DSP *Slices* yang dibutuhkan untuk mengimplementasikan koefisien filter *broadband* pada platform FPGA.

1.5 Manfaat

1.5.1 Bagi Peneliti

1. Meningkatkan pemahaman dan keterampilan dalam perancangan sistem *Active Noise Cancellation* (ANC), khususnya algoritma filter adaptif FXLMS dan FXNLMS, aritmetika *fixed-point*, serta implementasi dan sintesis HDL pada FPGA.

2. Menerapkan pengetahuan yang diperoleh selama pendidikan sarjana pada bidang pengolahan sinyal digital dan rekayasa elektronika.

1.5.2 Bagi Masyarakat Umum

1. Memberikan kontribusi pengetahuan mengenai perbandingan kinerja dan efisiensi komputasi algoritma FXLMS dan FXNLMS pada sistem ANC *headphone* berbasis FPGA, sebagai dasar kuantitatif dalam memilih algoritma ANC untuk perangkat tersemat berkapasitas terbatas.

[Halaman ini sengaja dikosongkan]

BAB II
TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Penelitian mengenai sistem ANC berbasis algoritma keluarga LMS telah dilakukan dengan objek, metode, dan platform implementasi yang beragam. Untuk memetakan posisi penelitian ini, lima penelitian terdahulu yang paling relevan dirangkum pada Tabel 2.1 berdasarkan objek atau data yang digunakan, metode, parameter dan hasil utama, serta keterkaitannya dengan penelitian ini.

Tabel 2.1: Hasil Penelitian Terdahulu

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
1.	Xie, Zhou & Xie (2022)	Derau acak (*random source*, f_s 60 kHz) pada model *feedforward* untuk *earphone* ANC	Simulasi FxLMS di MATLAB/Simulink, ditambah *low-pass filter* (LPF) FIR/Chebyshev orde 16 pada sinyal residu	Orde filter 5-30 dan *step size* 0,0001-1. Pada orde 8 dan μ = 0,5, STD derau 0,04797 turun menjadi 0,000778 (0,0003826 dengan LPF). Orde terlalu rendah tidak konvergen, sedangkan orde dan μ besar membuat sistem divergen	Menjadi acuan pengaruh orde filter dan *step size* terhadap kecepatan konvergensi dan stabilitas FxLMS pada sistem *headphone*. Masih berupa simulasi, implementasi *hardware* disebut sebagai penelitian lanjutan
2.	Han & Chen (2023)	Nada tunggal 200 Hz dan 300 Hz (±55 dB), serta perpindahan 200 Hz ke 300 Hz	*Hybrid fixed-filter* + FxLMS: FFT memilih koefisien pra-latih sebagai nilai awal FxLMS. Diimplementasikan pada mikrokontroler STM32H743	Orde 128, f_s 20 kHz, μ = 0,001. Reduksi ±15 dB (200 Hz) tidak berubah, tetapi waktu konvergensi 300 Hz turun dari 10 s menjadi 5 s. Pada simulasi (μ = 0,005), konvergensi dari ±50.000 sampel menjadi ±20.000 sampel	Menunjukkan kelemahan FxLMS konvensional (konvergensi lambat, beban komputasi) pada sistem digital nyata. Orde 128 dan f_s 20 kHz dapat menjadi pembanding parameter. Tidak membandingkan dengan NLMS dan tidak melaporkan penggunaan sumber daya

Tabel 2.1: Hasil Penelitian Terdahulu (lanjutan)

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
3.	Voltolini et al. (2023*)	Derau struktural pembangkit kogenerasi; 3 akselerometer sebagai referensi, 3 *shaker*, 1 mikrofon *error* pada jarak 40 m	FxNLMS *offline* di MATLAB (SISO hingga SIMO multi-referensi), lalu uji *real-time hardware-in-the-loop* pada DSP ADAU1466Z	Rentang 50-250 Hz, orde W = 256, *downsampling* 16× (48 kHz ke 3 kHz), μ = 0,1. Reduksi 3,6-4,0 dB(A) (terbaik J = 2, K = 2: 4,0 dB(A)). Hasil *real-time* identik dengan *offline*, konvergensi ±7 s	Salah satu dari sedikit *paper* yang menjalankan FxNLMS pada perangkat keras khusus secara *real-time*. Teknik *downsampling* untuk menekan beban komputasi relevan bagi FPGA. Bukan aplikasi *headphone*, tanpa pembanding FxLMS dan tanpa laporan sumber daya DSP
4.	Chen, Yang, Lai & Wei (2025)	Derau impulsif frekuensi rendah (tembakan artileri) untuk perlindungan pendengaran	FxLMS arsitektur *feedback* (AANC) + *audio-balance control circuit* (ABCC); simulasi MATLAB R2023b dan implementasi rangkaian elektronik	Simulasi orde 30-60, μ antara 10^{-3} dan 10^{-2}. Konfigurasi terbaik pada *hardware* adalah orde 30 dan $\mu = 10^{-3}$. Reduksi maksimum 21,8 dB, dan 10-20 dB di bawah 2 kHz	Menegaskan bahwa orde dan *step size* harus di-*tuning* ulang saat berpindah dari simulasi ke *hardware*. Berbasis rangkaian elektronik (op-amp TL074/OPA810), bukan FPGA. Tidak ada FxNLMS dan tidak ada analisis sumber daya digital
5.	Cesur et al. (2025*)	Derau jalan di kabin kendaraan listrik (rekaman lintasan uji, 2 mikrofon di posisi kepala pengemudi)	FxNLMS dengan *variable step-size*, pemodelan jalur sekunder *online* (PRBS + NLMS), *high-pass filter* dan *soft limiter*	f_s 44,1 kHz, *buffer* 256 sampel (±5,8 ms). Reduksi rata-rata 5,07 dB (8,28 dB di 250 Hz, 7,31 dB di 750 Hz), turun menjadi 2,4-3,6 dB di atas 1 kHz. Dibanding FxLMS standar: konvergensi lebih cepat dan reduksi lebih besar (disebut kualitatif, tanpa angka)	Paling dekat dengan topik penelitian ini karena menyatakan FxNLMS lebih unggul dari FxLMS dan cocok untuk sistem *embedded* dengan daya komputasi terbatas. Perbandingannya tidak kuantitatif, bukan di FPGA, dan tidak ada laporan sumber daya

Dua penelitian pertama mengkaji FXLMS pada tingkat simulasi dan mikrokontroler. Xie et al. [3] mensimulasikan FXLMS pada model *feedforward* untuk *earphone* ANC di MATLAB/Simulink dan menunjukkan bahwa orde filter dan *step-size* menentukan kestabilan algoritma. Pada orde 8 dan $\mu = 0{,}5$, standar deviasi derau turun dari 0,04797 menjadi 0,000778, sedangkan orde yang terlalu rendah gagal konvergen dan kombinasi orde serta μ yang besar membuat sistem divergen. Han [2] membawa FXLMS ke mikrokontroler STM32H743 dengan pendekatan *hybrid fixed-filter*. Reduksi sekitar 15 dB pada 200 Hz tidak berubah, tetapi waktu konvergensi pada 300 Hz berkurang dari 10 s menjadi 5 s. Kedua penelitian ini menegaskan bahwa FXLMS sensitif terhadap pemilihan μ dan cenderung lambat berkonvergensi, tetapi keduanya belum membandingkan FXLMS dengan varian ternormalisasinya dan tidak melaporkan

penggunaan sumber daya perangkat keras.

Dari sisi implementasi perangkat keras, Chen et al. [1] merealisasikan FXLMS dengan arsitektur *feedback* dalam bentuk rangkaian elektronik untuk derau impulsif frekuensi rendah dan memperoleh reduksi maksimum 21,8 dB. Penelitian tersebut juga menunjukkan bahwa rentang parameter pada simulasi, yaitu orde 30–60 dan μ antara 10^{-3} dan 10^{-2}, harus disesuaikan kembali menjadi orde 30 dan $\mu = 10^{-3}$ ketika dipindahkan ke *hardware*. Sementara itu, Voltolini et al. menjalankan FXNLMS secara *real-time* pada DSP ADAU1466Z untuk derau struktural pembangkit kogenerasi dan memperoleh hasil yang identik dengan simulasi *offline*, dengan reduksi 3,6–4,0 dB(A). Teknik *downsampling* 16× yang digunakan untuk menekan beban komputasi pada penelitian tersebut relevan bagi implementasi FPGA. Meskipun demikian, kedua penelitian ini tidak menyandingkan FXLMS dan FXNLMS secara langsung. Penelitian pertama tidak menggunakan FXNLMS, penelitian kedua tidak menggunakan FXLMS sebagai pembanding, dan keduanya tidak menyertakan analisis sumber daya digital.

Penelitian yang paling dekat dengan topik ini adalah Cesur et al. [4], yang menerapkan FXNLMS dengan *variable step-size* dan pemodelan jalur sekunder *online* pada kabin kendaraan listrik. Penelitian tersebut memperoleh reduksi rata-rata 5,07 dB, dengan reduksi terbesar 8,28 dB pada 250 Hz, dan reduksi yang turun menjadi 2,4–3,6 dB di atas 1 kHz. FXNLMS dinyatakan lebih cepat berkonvergensi dan menghasilkan reduksi yang lebih besar dibandingkan FXLMS standar, tetapi perbandingan tersebut hanya disampaikan secara kualitatif tanpa data kuantitatif, tidak diimplementasikan pada FPGA, dan tidak disertai laporan penggunaan sumber daya.

Berdasarkan Tabel 2.1, terdapat tiga hal yang belum terjawab oleh penelitian terdahulu. Pertama, sebagian besar penelitian hanya mengevaluasi satu algoritma, yaitu FXLMS [1, 2, 3] atau FXNLMS saja, sehingga perbandingan langsung kedua algoritma pada kondisi uji yang sama masih jarang dilakukan. Kedua, perbandingan FXLMS dan FXNLMS yang tersedia belum didukung data kuantitatif [4]. Ketiga, belum ada penelitian yang melaporkan biaya komputasi kedua algoritma dalam bentuk utilisasi sumber daya FPGA; data efisiensi perangkat keras FPGA yang tersedia berasal dari algoritma adaptif lain, seperti *Signed-Error* LMS [5]. Penelitian ini mengisi celah tersebut dengan membandingkan FXLMS dan FXNLMS secara kuantitatif pada sistem ANC *headphone feedforward* SISO, baik dari sisi kinerja peredaman, karakteristik konvergensi, maupun utilisasi sumber daya pada FPGA.

2.2 Dasar Teori

2.2.1 *Active Noise Cancellation* (ANC)

Active Noise Cancellation (ANC) adalah metode reduksi kebisingan yang bekerja dengan menghasilkan sinyal sekunder atau *anti-noise* secara aktif. ANC berbeda dengan metode pasif seperti peredam berbahan akustik. ANC bekerja berdasarkan prinsip interferensi destruktif: gelombang *anti-noise* dibuat dengan amplitudo setara dengan *noise* primer, tetapi dengan fasa berlawanan sekitar 180°. Ketika kedua gelombang bertemu pada titik kontrol, tekanan akustik residual menurun sehingga pendengar merasakan reduksi kebisingan [6].

Secara matematis, jika *noise* primer dinyatakan sebagai $d(n)$ dan sinyal *anti-noise* setelah melewati *secondary path* dinyatakan sebagai $y_s(n)$, maka sinyal residual pada mikrofon *error* adalah:

$$e(n) = d(n) + y_s(n) \tag{2.1}$$

Tujuan sistem ANC adalah membuat $y_s(n) \approx -d(n)$, sehingga $e(n) \approx 0$. Dalam praktiknya, pencapaian kondisi tersebut dipengaruhi oleh akurasi estimasi *secondary path*, karakteristik *noise*, kualitas transduser, serta latensi pemrosesan digital [6, 7].

Terdapat tiga topologi utama sistem ANC, yaitu:

1. *Feedforward* ANC menggunakan mikrofon referensi yang ditempatkan sebelum zona kontrol. Mikrofon ini mendeteksi *noise* sebelum *noise* mencapai telinga, kemudian sistem membentuk sinyal *anti-noise* berdasarkan sinyal referensi tersebut.

2. *Feedback* ANC hanya menggunakan mikrofon *error* di dalam zona kontrol. Sistem memperkirakan *noise* berdasarkan sinyal residual dan berusaha meminimalkannya, tetapi tanpa informasi awal dari sumber *noise*.

3. *Hybrid* ANC menggabungkan *feedforward* dan *feedback* untuk memperoleh reduksi yang lebih luas, terutama pada *noise* yang sebagian dapat diprediksi dan sebagian tidak dapat diprediksi. Namun, topologi ini membutuhkan lebih banyak sensor, aktuator, dan kompleksitas kendali [6, 8].

Penelitian ini memilih topologi *feedforward Single-Input Single-Output* (SISO) karena sesuai dengan konfigurasi *headphone* sederhana. Satu mikrofon referensi ditempatkan pada bagian luar *earcup* untuk menangkap *noise* lingkungan, satu mikrofon *error* ditempatkan di dalam *earcup* untuk mengukur sinyal residual, dan satu *speaker* digunakan sebagai aktuator penghasil *anti-noise*. Konfigurasi ini hanya membutuhkan satu jalur pemrosesan utama sehingga lebih efisien untuk diimplementasikan pada FPGA dengan sumber daya terbatas [7, 8].

Pemilihan *feedforward* juga didasarkan pada aspek kausalitas. Pada sistem *feedforward*, mikrofon referensi harus menerima *noise* lebih awal daripada mikrofon *error*. Waktu rambat akustik dari mikrofon referensi ke mikrofon *error* memberikan margin waktu bagi FPGA untuk melakukan akuisisi data, komputasi filter adaptif, dan pengiriman sinyal ke DAC. Jika total latensi digital melebihi *delay* akustik tersebut, *anti-noise* tiba setelah *noise* primer sehingga interferensi destruktif tidak lagi efektif [6, 7].

Topologi *feedback* dan *hybrid* tetap relevan sebagai pembanding. *Feedback* ANC dapat meredam komponen *noise* periodik yang dapat diprediksi, tetapi lebih sensitif terhadap umpan balik akustik dan berpotensi menimbulkan osilasi. *Hybrid* ANC memberikan performa lebih baik pada kondisi tertentu, tetapi tidak dipilih karena kebutuhan perangkat keras dan kompleksitas algoritmanya lebih tinggi [6, 9].

2.2.2 Filter Adaptif dan Algoritma LMS

Filter adaptif adalah filter yang koefisiennya diperbarui secara bertahap untuk meminimalkan galat antara keluaran sistem dan sinyal yang diinginkan. Pada sistem ANC, filter adaptif berfungsi membentuk sinyal *anti-noise* yang sesuai dengan karakteristik *noise* primer dan *secondary path*. Filter yang umum digunakan adalah *Finite Impulse Response* (FIR) karena stabil secara inheren, relatif mudah diimplementasikan, dan mendukung struktur perkalian-akumulasi yang efisien pada FPGA [6, 10].

Keluaran filter FIR adaptif dengan panjang L dinyatakan sebagai:

$$y(n) = \sum_{k=0}^{L-1} w_k(n)\, x(n-k) \tag{2.2}$$

dengan $x(n)$ adalah sinyal masukan, $w_k(n)$ adalah koefisien filter pada sampel ke-n, dan $y(n)$ adalah sinyal *anti-noise*. Dalam bentuk vektor, persamaan tersebut menjadi:

$$y(n) = \mathbf{w}^T(n)\, \mathbf{x}(n) \tag{2.3}$$

dengan:

$$\mathbf{w}(n) = \begin{bmatrix} w_0(n) & w_1(n) & \cdots & w_{L-1}(n) \end{bmatrix}^T \tag{2.4}$$

$$\mathbf{x}(n) = \begin{bmatrix} x(n) & x(n-1) & \cdots & x(n-L+1) \end{bmatrix}^T \tag{2.5}$$

Tujuan adaptasi adalah meminimalkan fungsi biaya *Mean Square Error* (MSE):

$$J(n) = E\left[e^2(n)\right] \tag{2.6}$$

dengan $e(n)$ sebagai sinyal galat. Karena nilai statistik jangka panjang sulit dihitung secara langsung pada sistem *real-time*, algoritma *Least Mean Square* (LMS) menggunakan estimasi gradien sesaat. Pembaruan koefisien LMS dinyatakan sebagai:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\, e(n)\, \mathbf{x}(n) \tag{2.7}$$

dengan μ adalah *step-size* [6, 11].

Step-size merupakan parameter yang menentukan keseimbangan antara kecepatan konvergensi dan kestabilan. Nilai μ yang besar membuat koefisien bergerak cepat menuju solusi optimum, tetapi dapat menyebabkan osilasi atau divergensi apabila terlalu besar. Sebaliknya, μ yang kecil menghasilkan adaptasi yang stabil tetapi lambat, terutama ketika karakteristik *noise* berubah cepat. Oleh sebab itu, pemilihan μ harus mempertimbangkan panjang filter, daya sinyal masukan, dan tingkat variasi lingkungan akustik [6, 11].

2.2.3 Algoritma FXLMS

Pada sistem ANC, keluaran filter adaptif tidak langsung membentuk interferensi destruktif pada mikrofon *error*. Sinyal *anti-noise* harus melewati *secondary path*, yaitu rangkaian dari DAC, penguat audio, *speaker*, rambatan akustik di dalam *earcup*, hingga mikrofon *error*. *Secondary path* memiliki *delay* waktu dan respons frekuensi yang tidak ideal. Akibatnya, algoritma LMS standar menjadi kurang tepat karena sinyal referensi yang digunakan untuk pembaruan koefisien belum memperhitungkan pengaruh *secondary path* [6, 7].

Algoritma *Filtered-x Least Mean Square* (FXLMS) mengatasi masalah tersebut dengan memfilter sinyal referensi menggunakan estimasi *secondary path*. Diagram blok FXLMS *feed-forward* SISO terdiri atas filter adaptif $W(z)$, estimasi *secondary path* $\hat{S}(z)$, *speaker*, mikrofon

error, dan pembentuk sinyal *anti-noise*. Sinyal referensi $x(n)$ dari mikrofon luar diproses oleh $W(z)$ untuk menghasilkan $y(n)$, kemudian $x(n)$ juga difilter oleh $\hat{S}(z)$ sehingga diperoleh $x'(n)$ untuk pembaruan koefisien [6, 7, 9].

Definisi sinyal pada FXLMS adalah sebagai berikut:

- $x(n)$: sinyal *noise* referensi dari mikrofon di luar *earcup*.
- $x'(n)$: sinyal referensi yang telah difilter oleh estimasi *secondary path*.
- $y(n)$: keluaran filter adaptif yang dikirim ke *speaker*.
- $e(n)$: sinyal residual dari mikrofon *error* di dalam *earcup*.
- $\mathbf{w}(n)$: vektor koefisien filter adaptif FIR.

Sinyal referensi terfilter dinyatakan sebagai:

$$x'(n) = \hat{s}(n) * x(n) \tag{2.8}$$

dengan $\hat{s}(n)$ adalah *impulse response* estimasi *secondary path*. Pembaruan koefisien FXLMS dengan *step-size* tetap adalah [6, 7]:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\, e(n)\, \mathbf{x}'(n) \tag{2.9}$$

dengan:

$$\mathbf{x}'(n) = \begin{bmatrix} x'(n) & x'(n-1) & \cdots & x'(n-L+1) \end{bmatrix}^T \tag{2.10}$$

Syarat kestabilan FXLMS secara umum dapat dinyatakan sebagai:

$$0 < \mu < \frac{2}{L P_{x'}} \tag{2.11}$$

dengan $P_{x'}$ adalah daya sinyal referensi terfilter dan L adalah panjang filter adaptif. Persamaan ini memperlihatkan bahwa nilai μ yang aman bergantung pada daya sinyal. Bila daya $x'(n)$ meningkat, μ harus diperkecil agar pembaruan koefisien tetap stabil. Sebaliknya, saat daya sinyal menurun, nilai μ yang tetap menyebabkan konvergensi menjadi lambat [4, 6, 11].

Kelemahan utama FXLMS adalah penggunaan *step-size* tetap. Pada lingkungan dengan variasi level *noise* yang besar, misalnya saat pengguna berpindah dari ruangan sunyi ke lingkungan lalu lintas, FXLMS dapat kehilangan keseimbangan antara kecepatan adaptasi dan kestabilan. Oleh karena itu, diperlukan normalisasi *step-size* terhadap daya sinyal referensi [4, 11].

2.2.4 Algoritma FXNLMS

Algoritma *Filtered-x Normalized Least Mean Square* (FXNLMS) merupakan pengembangan FXLMS yang menormalisasi *step-size* terhadap daya sinyal referensi terfilter. Tujuannya adalah menjaga besaran pembaruan koefisien tetap proporsional terhadap level sinyal masukan. Dengan demikian, algoritma dapat tetap stabil pada sinyal kuat dan tetap mampu berkonvergensi pada sinyal lemah [4, 11].

Pembaruan koefisien FXNLMS dinyatakan sebagai:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\mu}{\hat{P}_{x'}(n) + \varepsilon}\, e(n)\, \mathbf{x}'(n) \tag{2.12}$$

dengan $\hat{P}_{x'}(n)$ adalah estimasi daya sinyal referensi terfilter dan ε adalah konstanta regularisasi kecil. Konstanta ε mencegah pembagian dengan nilai mendekati nol ketika sinyal referensi sangat lemah atau senyap. Tanpa ε, pembagian tersebut dapat menghasilkan nilai pembaruan yang sangat besar dan menyebabkan ketidakstabilan numerik [4, 11].

Estimasi daya dapat dilakukan menggunakan metode rekursif:

$$\hat{P}_{x'}(n) = \lambda\, \hat{P}_{x'}(n-1) + (1-\lambda)\, x'^2(n) \tag{2.13}$$

dengan $0 < \lambda < 1$ adalah faktor peluruhan. Nilai λ yang mendekati 1 menghasilkan estimasi daya yang lebih halus dan stabil, sedangkan nilai λ yang lebih kecil membuat estimasi lebih responsif terhadap perubahan sinyal. Metode ini hanya membutuhkan penyimpanan satu nilai daya sebelumnya, sehingga lebih efisien dibanding metode *sliding window* yang harus menyimpan sejumlah sampel sebelumnya [4, 11].

Dari sisi implementasi FPGA, FXNLMS menambahkan operasi pembagian atau perkalian dengan invers daya. Operasi ini lebih mahal dibanding operasi perkalian dan penjumlahan pada FXLMS. Untuk menghindari pembagi yang besar, dapat digunakan blok *reciprocal*, tabel pencarian, atau metode iteratif seperti Newton-Raphson. Namun, setiap pendekatan menambah kebutuhan sumber daya, latensi, atau kompleksitas kontrol *timing* [7, 9].

Tabel 2.2: Perbandingan FXLMS dan FXNLMS

Aspek	**FXLMS**	**FXNLMS**
Step-size efektif	Tetap, μ	Dinormalisasi, μ / ($\hat{P}_{x'} + \varepsilon$)
Operasi utama	Perkalian dan akumulasi	Perkalian, akumulasi, estimasi daya, dan pembagian
Konvergensi	Bergantung pada daya $x'(n)$	Lebih konsisten terhadap variasi daya sinyal
Kestabilan	Perlu μ kecil saat daya *input* besar	Lebih *robust* terhadap perubahan level *input*
Kompleksitas FPGA	Lebih rendah	Lebih tinggi karena pembagi atau blok *reciprocal*
Kelemahan	Lambat saat sinyal lemah, berisiko tidak stabil saat sinyal kuat	Biaya komputasi dan konsumsi sumber daya lebih besar

Berdasarkan perbandingan tersebut, FXLMS cocok untuk implementasi awal yang menekankan efisiensi perangkat keras, sedangkan FXNLMS lebih sesuai bila lingkungan *noise* memiliki variasi amplitudo yang signifikan [4, 7, 11].

2.2.5 Representasi *Fixed-Point*

Implementasi FPGA untuk pemrosesan audio dan filter adaptif umumnya menggunakan aritmetika *fixed-point* karena lebih hemat sumber daya, memiliki latensi rendah, dan dapat dipetakan langsung ke DSP *slice*. Format *fixed-point* yang umum digunakan adalah Q*m.n*, dengan *m* bit bagian integer, *n* bit bagian fraksi, dan biasanya satu bit tanda. Total *wordlength* adalah:

$$B = m + n + 1 \tag{2.14}$$

Sebagai contoh, format Q1.15 menggunakan satu bit tanda dan 15 bit fraksi, sehingga rentang nilai yang dapat direpresentasikan adalah [9, 10]:

$$-1 \leq x \leq 1 - 2^{-15} \tag{2.15}$$

Proses kuantisasi menyebabkan galat karena nilai kontinu atau hasil perhitungan harus dibulatkan atau dipotong agar sesuai dengan jumlah bit yang tersedia. Galat ini muncul pada sampel audio dari I2S, koefisien filter, hasil perkalian DSP, hasil akumulasi MAC, dan nilai daya pada FXNLMS. Semakin kecil *wordlength*, semakin besar galat kuantisasi dan semakin besar potensi penurunan kualitas reduksi *noise* [7, 10].

Overflow terjadi apabila hasil operasi melebihi rentang representasi *fixed-point*. Pada sistem audio, *overflow* dapat menimbulkan distorsi keras atau *clipping*. Salah satu penanganan yang umum adalah *saturation*, yaitu membatasi hasil operasi pada nilai maksimum atau minimum yang dapat direpresentasikan. Selain itu, *scaling* dapat diterapkan pada koefisien, sinyal masukan, atau hasil *intermediate* agar nilai tetap berada dalam rentang aman [9, 10].

Pada filter adaptif, presisi terbatas memiliki dampak yang lebih serius dibanding filter statis. Pertama, galat kuantisasi pada $e(n)$, $x'(n)$, dan μ menyebabkan arah pembaruan koefisien menjadi tidak sempurna. Kedua, jika nilai $\mu\, e(n)\, x'(n)$ lebih kecil daripada resolusi koefisien, hasil penjumlahan tidak mengubah nilai koefisien. Kondisi ini disebut *coefficient stalling* atau berhentinya pembaruan koefisien. Akibatnya, filter tidak dapat mencapai solusi optimum meskipun secara matematis algoritma seharusnya masih beradaptasi [7, 9].

Fixed-point dipilih dibanding *floating-point* karena FPGA Artix-7 memiliki DSP *slice* yang dioptimasi untuk operasi MAC *fixed-point*. *Floating-point* membutuhkan lebih banyak LUT, register, dan *pipeline* sehingga meningkatkan penggunaan sumber daya serta konsumsi daya. Untuk sistem ANC *headphone* dengan kebutuhan *real-time* dan latensi rendah, *fixed-point* memberikan keseimbangan yang lebih baik antara akurasi, kecepatan, dan efisiensi perangkat keras [9, 10, 12].

2.2.6 *Field-Programmable Gate Array* (FPGA)

Field-Programmable Gate Array (FPGA) adalah perangkat logika *programmable* yang dapat dikonfigurasi ulang untuk membentuk rangkaian digital sesuai kebutuhan. Berbeda dengan mikrokontroler yang mengeksekusi instruksi secara berurutan, FPGA dapat menjalankan banyak operasi secara paralel. Karakteristik ini sangat sesuai untuk filter FIR, konvolusi, pembaruan LMS, dan antarmuka I2S yang harus diproses dalam satu periode *sampling* [7, 10].

Elemen dasar FPGA meliputi:

1. *Look-Up Table* (LUT): mengimplementasikan fungsi logika kombinasional seperti *multiplexer*, *adder*, *comparator*, dan kontrol *finite-state machine*.

2. *Flip-Flop*: menyimpan satu bit data pada tepi *clock* dan membentuk register, *counter*, serta *pipeline*.

3. DSP *slice*: melakukan operasi *multiply-accumulate* (MAC) secara efisien untuk filter FIR, konvolusi, dan pembaruan adaptif.

4. *Block RAM* (BRAM): menyimpan *delay line* filter, koefisien adaptif, *buffer* I2S, dan data internal lainnya [12, 13].

Perangkat yang digunakan adalah Artix-7 XC7A35T pada *board* Arty A7-35T. FPGA ini memiliki 5.200 *slice*, 41.600 *flip-flop*, 1.800 Kbit *block RAM*, dan 90 DSP *slice*. Setiap *slice* terdiri atas empat LUT enam-*input* dan delapan *flip-flop*. Spesifikasi tersebut memadai untuk implementasi filter FIR adaptif berukuran sedang, kontrol I2S, serta pemrosesan FXLMS atau FXNLMS secara *real-time* [12, 13].

DSP *slice* menjadi komponen utama dalam implementasi filter adaptif karena operasi FXLMS didominasi oleh perkalian koefisien dengan sampel sinyal dan akumulasi hasil perkalian. BRAM digunakan untuk menyimpan *delay line* $x(n)$ dan $x'(n)$, sedangkan register *flip-flop* menyimpan koefisien $w(n)$, sinyal *error*, dan sinyal kendali *pipeline*. Pembagian pada FXNLMS dapat diimplementasikan menggunakan LUT, blok *reciprocal*, atau modul pembagi yang di-*pipeline* [7, 9].

Alur perancangan HDL dimulai dengan penulisan deskripsi perangkat keras menggunakan VHDL atau Verilog. Desain kemudian diverifikasi melalui *testbench* dan simulasi fungsional untuk memastikan keluaran sesuai spesifikasi. Setelah itu, desain disintesis menggunakan Vivado untuk menghasilkan *netlist* logika, dilanjutkan dengan implementasi, analisis *timing*, dan pembangkitan *bitstream*. Pada sistem ANC, *timing closure* sangat penting karena seluruh jalur pemrosesan harus selesai sebelum batas satu periode *sampling* terlampaui [9, 10].

2.2.7 Antarmuka Audio I2S

Inter-IC Sound (I2S) adalah protokol komunikasi digital yang digunakan untuk mentransfer data audio antarperangkat digital, misalnya mikrofon MEMS, FPGA, dan DAC. I2S menggunakan tiga jalur utama:

1. SCK atau BCLK: *serial clock* untuk sinkronisasi bit data.

2. WS atau LRCLK: *word select* untuk memilih kanal kiri atau kanan.

3. SD atau DOUT/DIN: jalur data serial yang membawa sampel audio [14, 15].

Pada sistem ini, INMP441 berfungsi sebagai mikrofon MEMS digital dengan keluaran I2S. Modul ini menghasilkan data PCM 24-bit dan mendukung frekuensi *sampling* sekitar 8 kHz hingga 48 kHz. FPGA berperan sebagai I2S *master* yang menghasilkan SCK dan WS, kemudian menerima data audio dari INMP441 melalui jalur SD. Data tersebut diubah menjadi sinyal referensi $x(n)$ yang digunakan oleh filter adaptif [14, 15].

PCM5102A berperan sebagai DAC audio yang menerima data digital dari FPGA melalui I2S dan mengubahnya menjadi sinyal analog untuk *speaker*. Keluaran FPGA berupa sinyal *anti-noise* $y(n)$ dikirim ke PCM5102A, kemudian diubah menjadi gelombang akustik oleh *speaker* di dalam *earcup*. Mikrofon *error* di dalam *earcup* kemudian mengukur sinyal residual $e(n)$ yang digunakan untuk memperbarui koefisien filter adaptif [10, 16].

Frekuensi *sampling* menentukan waktu yang tersedia untuk memproses satu sampel. Periode *sampling* adalah:

$$T_s = \frac{1}{f_s} \tag{2.16}$$

Pada $f_s = 48$ kHz, periode *sampling* adalah sekitar 20,83 μs. Seluruh proses, termasuk penerimaan I2S dari INMP441, pemrosesan filter FIR, pembaruan koefisien adaptif, dan pengiriman data ke PCM5102A, harus selesai dalam waktu tersebut agar sistem tetap *real-time* [7, 15, 16].

Latensi total sistem dapat dinyatakan sebagai:

$$T_{\text{latency}} \approx T_{\text{I2S}} + T_{\text{FIR}} + T_{\text{adaptif}} + T_{\text{DAC}} \tag{2.17}$$

Latensi ini harus lebih kecil daripada waktu rambat akustik dari mikrofon referensi di luar *earcup* ke mikrofon *error* di dalam *earcup*. Jika tidak, *anti-noise* tiba terlambat dan interferensi destruktif tidak optimal. Oleh karena itu, implementasi FPGA dengan arsitektur paralel dan *pipelined* menjadi penting untuk memenuhi kendala kausalitas *feedforward* ANC [6, 7, 8].

BAB III
METODOLOGI

3.1 Metode yang Digunakan

Penelitian ini menggunakan metode eksperimen kuantitatif yang terdiri atas tiga tahap utama, yaitu simulasi dan evaluasi kinerja peredaman algoritma FXLMS dan FXNLMS, analisis pengaruh normalisasi *step-size* terhadap karakteristik konvergensi, serta implementasi dan evaluasi efisiensi komputasi pada FPGA. Ketiga tahap tersebut disusun secara berurutan agar penelitian memiliki alur yang sistematis. Penelitian dimulai dari perbandingan kemampuan peredaman kedua algoritma pada sistem ANC *headphone*, dilanjutkan dengan penelusuran penyebab perbedaan kinerja pada mekanisme normalisasi *step-size*, dan diakhiri dengan pengukuran biaya komputasi yang diperlukan untuk memperoleh kinerja tersebut pada platform FPGA. Seluruh pengujian dibatasi pada sistem ANC *feedforward single-channel* (SISO) yang merepresentasikan satu sisi *earpiece* pada *headphone*.

3.1.1 Metode Simulasi dan Evaluasi Kinerja Peredaman

Tahap pertama dalam penelitian ini adalah perancangan model sistem ANC *feedforward* SISO pada perangkat lunak (MATLAB atau Python) untuk membandingkan kemampuan peredaman algoritma FXLMS dan FXNLMS. Model sistem mencakup jalur primer dari mikrofon referensi ke mikrofon *error*, jalur sekunder dari *speaker* ke mikrofon *error*, estimasi jalur sekunder yang digunakan untuk memfilter sinyal referensi (*filtered-x*), serta filter adaptif berstruktur FIR. Dua algoritma yang diimplementasikan pada model tersebut adalah FXLMS standar dengan *step-size* tetap dan FXNLMS dengan *step-size* yang dinormalisasi oleh daya sinyal referensi terfilter. Sinyal uji yang digunakan berupa *white noise*, *pink noise*, dan rekaman bising lingkungan nyata (*real-world ambient noise*) pada pita frekuensi rendah di bawah 1000 Hz hingga 2000 Hz.

Kedua algoritma diuji pada kondisi yang sama, yaitu sinyal uji, model jalur primer dan sekunder, panjang filter, frekuensi *sampling*, dan kondisi awal koefisien filter, agar perbandingan berlangsung adil dan terukur. Nilai *step-size* pada FXLMS ditentukan melalui penelusuran beberapa nilai (*tuning*) sehingga algoritma tersebut dibandingkan pada kondisi terbaiknya, bukan pada nilai yang kebetulan merugikan. Parameter evaluasi yang digunakan pada tahap ini adalah peredaman kebisingan (*noise reduction*) yang dinyatakan dalam SNR (dB), kecepatan konvergensi, dan *steady-state error*. Hasil evaluasi tersebut digunakan untuk menjawab seberapa besar keunggulan FXNLMS dibandingkan FXLMS standar dalam meredam kebisingan *broadband*.

Pada tahap awal ini, penelitian juga menetapkan dugaan awal terhadap karakteristik masing-masing algoritma. FXLMS diduga memiliki struktur operasi yang lebih sederhana dan hemat sumber daya, tetapi kinerjanya bergantung pada pemilihan *step-size* secara manual dan rentan terhadap variasi daya sinyal kebisingan. FXNLMS diduga mampu menjaga kecepatan konvergensi dan kestabilan pada daya sinyal yang berfluktuasi karena *step-size* menyesuaikan diri secara otomatis, tetapi membutuhkan estimasi daya sinyal dan operasi pembagian tam-

bahan pada setiap iterasi. Berdasarkan dugaan tersebut, FXNLMS diperkirakan unggul dari sisi kinerja peredaman, sedangkan FXLMS diperkirakan unggul dari sisi efisiensi komputasi, meskipun kesimpulan akhir tetap ditentukan berdasarkan hasil eksperimen.

Tabel 3.1: Rancangan Awal Algoritma dan Dugaan Awal

Algoritma	**Bentuk Umum Rancangan**	**Dugaan Awal**
FXLMS	Filter adaptif FIR dengan *step-size* tetap (μ). Pembaruan koefisien: $\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\, e(n)\, \mathbf{x}'(n)$, dengan $x'(n)$ adalah sinyal referensi yang difilter melalui estimasi jalur sekunder	Operasi lebih sederhana dan hemat sumber daya, tetapi kinerjanya bergantung pada *tuning* μ dan rentan terhadap variasi daya sinyal
FXNLMS	Struktur sama dengan FXLMS, dengan *step-size* dinormalisasi oleh daya sinyal referensi terfilter. Pembaruan koefisien: $\mathbf{w}(n+1) = \mathbf{w}(n) + \dfrac{\mu\, e(n)\, \mathbf{x}'(n)}{\varepsilon + \|\mathbf{x}'(n)\|^2}$	Konvergensi lebih cepat dan lebih tahan terhadap fluktuasi daya sinyal, tetapi memerlukan estimasi daya dan operasi pembagian tambahan

Sebagai dasar pelaksanaan eksperimen, kedua algoritma dibangun dengan arsitektur filter yang identik, sehingga perbedaan hasil hanya berasal dari mekanisme pembaruan koefisien, yaitu *step-size* tetap pada FXLMS dan *step-size* ternormalisasi pada FXNLMS.

3.1.2 Metode Analisis Pengaruh Normalisasi *Step-Size*

Tahap kedua adalah analisis pengaruh mekanisme normalisasi *step-size* pada FXNLMS terhadap kecepatan konvergensi dan *steady-state error*. Analisis dilakukan pada kerangka simulasi yang sama dengan tahap pertama, tetapi dengan skenario yang secara khusus memunculkan fluktuasi daya sinyal *broadband*, misalnya perubahan tingkat daya kebisingan secara tiba-tiba selama simulasi berlangsung. Skenario ini dipilih karena kelemahan utama FXLMS dengan *step-size* tetap muncul ketika daya sinyal referensi berubah, sehingga menimbulkan dilema antara kecepatan konvergensi dan kestabilan.

Pada skenario tersebut, FXLMS diuji dengan beberapa nilai *step-size* untuk menunjukkan hubungan antara kecepatan konvergensi dan kestabilan, sedangkan FXNLMS diuji pada konfigurasi normalisasi yang sama pada setiap skenario. Perilaku kedua algoritma diamati melalui kurva pembelajaran (*learning curve*) dari daya sinyal *error* terhadap jumlah iterasi. Kecepatan konvergensi diukur dari jumlah iterasi yang dibutuhkan hingga *error* mencapai kisaran nilai tunaknya, sedangkan *steady-state error* diukur dari rata-rata daya sinyal *error* setelah algoritma konvergen. Setiap skenario diulang beberapa kali dan hasilnya dirata-ratakan untuk mengurangi pengaruh sifat acak sinyal uji. Dengan demikian, fokus utama tahap ini adalah menentukan apakah normalisasi *step-size* benar-benar mempercepat konvergensi dan menurunkan *steady-state error* dibandingkan *step-size* tetap.

3.1.3 Metode Implementasi pada FPGA dan Evaluasi Efisiensi Komputasi

Tahap ketiga adalah evaluasi efisiensi komputasi FXLMS dan FXNLMS, yang terdiri atas analisis kompleksitas operasi dan implementasi pada FPGA. Pada bagian pertama, jumlah op-

erasi perkalian, penjumlahan, dan pembagian per iterasi dihitung secara analitis untuk kedua algoritma. Perhitungan ini bertujuan menunjukkan tambahan beban komputasi pada FXNLMS, yaitu estimasi daya sinyal referensi terfilter dan satu operasi pembagian pada setiap iterasi.

Pada bagian kedua, kedua algoritma dirancang menggunakan *hardware description language* (HDL) dengan aritmetika *fixed-point* dan disintesis menggunakan Vivado pada *board* Digilent Arty A7-35T yang menggunakan FPGA Xilinx Artix-7 XC7A35T. Dalam penelitian ini FPGA diposisikan sebagai platform implementasi untuk mengukur biaya komputasi algoritma, bukan sebagai fokus perancangan perangkat keras tingkat rendah. Panjang filter, lebar bit data, dan frekuensi *clock* dibuat sama pada kedua arsitektur agar perbedaan utilisasi sumber daya hanya berasal dari perbedaan algoritma. Parameter evaluasi pada bagian ini adalah utilisasi sumber daya logika yang diperoleh dari laporan sintesis dan implementasi, yaitu LUT, *Flip-Flop*, memori, dan DSP *Slices*.

Untuk memverifikasi bahwa rancangan dapat berfungsi pada perangkat sebenarnya, hasil implementasi diuji pada prototipe ANC *headphone* yang terdiri atas mikrofon referensi dan mikrofon *error* berbasis INMP441, DAC audio PCM5102A, penguat *headphone* TPA6132A2, serta *speaker driver*. Sumber kebisingan berupa *white noise* dan *pink noise* dari *speaker* aktif, dan tingkat peredamannya diukur secara independen menggunakan mikrofon ukur. Pengujian pada prototipe ini berfungsi sebagai verifikasi implementasi, sedangkan evaluasi kinerja algoritma tetap bertumpu pada hasil simulasi sesuai batasan penelitian. Tahap ini bertujuan memberikan gambaran tentang harga komputasi yang harus dibayar untuk memperoleh keunggulan FXNLMS dibandingkan FXLMS standar.

3.2 Bahan dan Peralatan yang Digunakan

Bahan dan peralatan yang digunakan dalam penelitian ini meliputi seluruh komponen yang mendukung proses penelitian secara menyeluruh, baik pada tahap simulasi algoritma, analisis normalisasi *step-size*, maupun implementasi pada FPGA. Bahan yang digunakan mencakup sinyal uji dan referensi ilmiah, sedangkan peralatan yang digunakan berupa perangkat keras dan perangkat lunak yang menunjang proses akuisisi sinyal, simulasi, sintesis, pengukuran, dan analisis data. Rincian bahan dan peralatan tersebut disajikan pada Tabel 3.2.

Tabel 3.2: Bahan dan Peralatan yang Digunakan

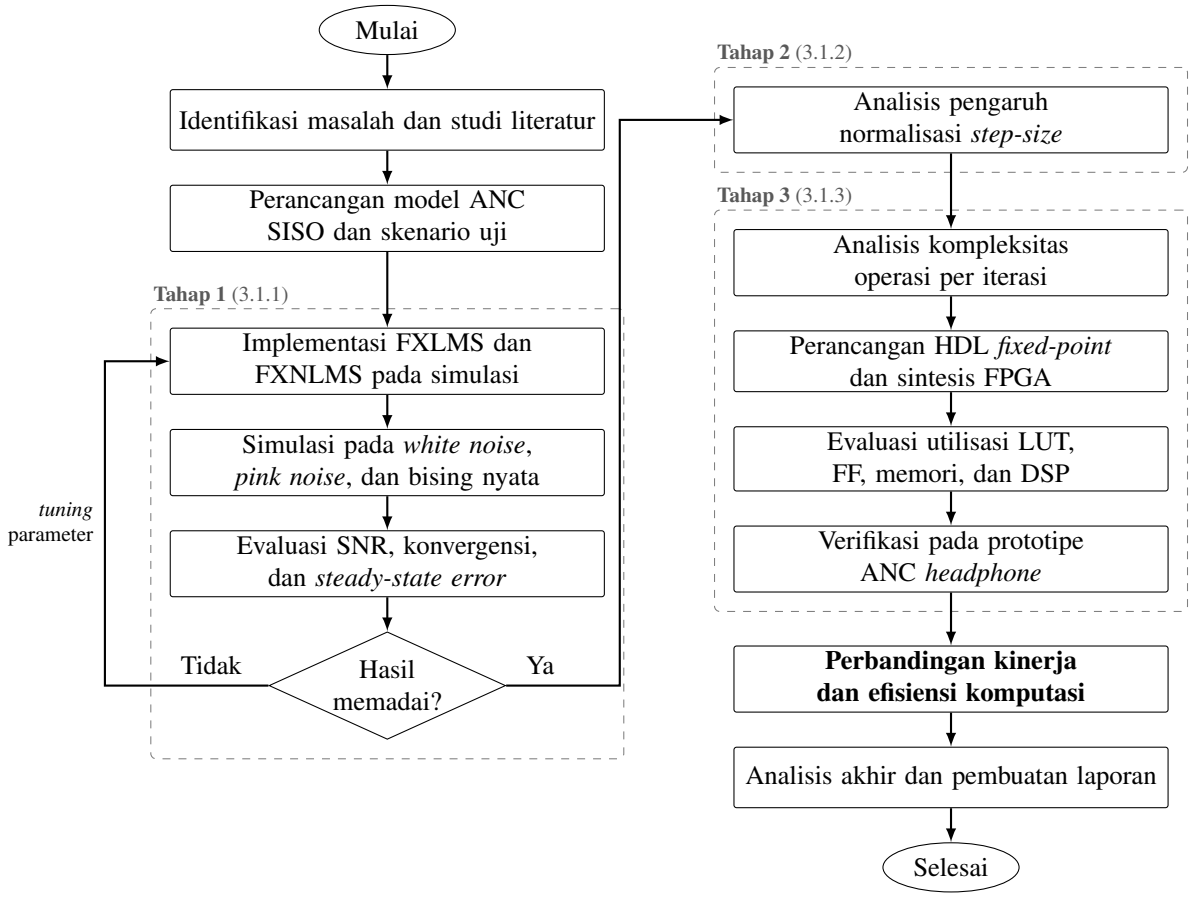
No.	**Nama Peralatan / Komponen**	**Spesifikasi**	**Fungsi**
1.	*Board* FPGA	Digilent Arty A7-35T, Xilinx Artix-7 XC7A35T, 20.800 LUT, 41.600 FF, 90 DSP48E1, 100 MHz	Platform implementasi algoritma FXLMS dan FXNLMS
2.	Mikrofon Referensi	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal kebisingan di luar *earcup*
3.	Mikrofon *Error*	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal residu di dalam *earcup*, dekat telinga

Tabel 3.2: Bahan dan Peralatan yang Digunakan (lanjutan)

No.	Nama Peralatan / Komponen	Spesifikasi	Fungsi
4.	DAC Audio	PCM5102A, antarmuka I2S, 32-bit, 112 dB SNR	Konversi sinyal *anti-noise* digital menjadi analog
5.	Penguat *Headphone*	TPA6132A2, stereo, *class*-G, 2,3-5,5 V	Penguatan sinyal analog untuk menggerakkan *speaker*
6.	*Speaker Driver*	Dinamis 40 mm, 32 Ω, 2 unit (1 aktif, 1 cadangan)	Pembangkit sinyal *anti-noise*
7.	Regulator Tegangan Rendah *Noise*	LP5907, LDO 3,3 V, 250 mA, *noise* 6,5 μVrms	Catu daya mikrofon bebas riak
8.	Mikrofon Ukur	MIC-L20 *Clip On Microphone* USB	Pengukuran independen tingkat peredaman kebisingan (dB)
9.	Sumber Kebisingan	*Speaker* aktif/*studio monitor*	Pembangkit *white noise* dan *pink noise* uji
10.	Komputer / Laptop	Windows 11/Linux	Menjalankan perangkat lunak sintesis HDL, simulasi, dan analisis data
11.	Vivado Design Suite	Perangkat lunak sintesis dan implementasi FPGA Xilinx	Sintesis HDL serta pengambilan laporan utilisasi LUT, *Flip-Flop*, memori, dan DSP *Slices*
12.	MATLAB / Python	Perangkat lunak simulasi dan analisis numerik	Simulasi sistem ANC, implementasi FXLMS dan FXNLMS, serta perhitungan metrik kinerja
13.	Sinyal Uji	*White noise*, *pink noise*, dan rekaman bising lingkungan nyata	Sinyal kebisingan referensi untuk simulasi dan pengujian prototipe
14.	Referensi jurnal dan artikel ilmiah	Jurnal, prosiding, dan artikel terkait ANC, FXLMS, FXNLMS, dan implementasi FPGA	Dasar teoritis dalam pemilihan algoritma, parameter evaluasi, dan implementasi pada FPGA

3.3 Urutan Pelaksanaan Penelitian

Urutan pelaksanaan penelitian ini digambarkan melalui diagram alir pada Gambar 3.1. Penelitian dimulai dari identifikasi masalah dan studi literatur, dilanjutkan dengan perancangan model sistem ANC dan simulasi kedua algoritma, kemudian analisis pengaruh normalisasi *step-size*. Setelah hasil simulasi dinilai memadai, penelitian dilanjutkan dengan analisis kompleksitas operasi, implementasi pada FPGA, dan verifikasi pada prototipe, lalu diakhiri dengan analisis akhir dan penyusunan laporan.



Gambar 3.1: Diagram Alir Pelaksanaan Penelitian

3.4 Jadwal Pelaksanaan

Penelitian direncanakan berlangsung selama enam bulan. Rincian kegiatan dan alokasi waktunya disajikan pada Tabel 3.3.

Tabel 3.3: Jadwal Pelaksanaan

No.	Jenis Kegiatan	Bulan 1	Bulan 2	Bulan 3	Bulan 4	Bulan 5	Bulan 6
1	Identifikasi masalah						
2	Studi literatur						
3	Perancangan model sistem ANC dan skenario uji						
4	Implementasi algoritma FXLMS dan FXNLMS pada simulasi						
5	Simulasi dan evaluasi kinerja peredaman						
6	Analisis pengaruh normalisasi *step-size*						
7	Perancangan dan implementasi algoritma pada FPGA						
8	Sintesis dan evaluasi utilisasi sumber daya						
9	Pengujian prototipe dan pengukuran peredaman						
10	Perbandingan efisiensi komputasi FXLMS dan FXNLMS						
11	Analisis akhir						
12	Pembuatan laporan						

DAFTAR PUSTAKA

[1] T.-Y. Chen, J.-H. Yang, C.-L. Lai, and C.-T. Wei, "Low-frequency active noise control system based on feedback FXLMS," *Electronics*, vol. 14, no. 7, 2025, Art. no. 1442.

[2] Y. Han, *Active noise control system based on hybrid fixed filter method and FxLMS algorithm*, Nanjing University of Aeronautics and Astronautics, 2023.

[3] X. H. Xie, L. N. Zhou, and Y. J. Xie, *Design and simulation of active noise cancelling earphone system based on FXLMS algorithm*, Xi'an University of Posts and Telecommunications, 2022.

[4] A. Cesur, A. E. Canbey, H. Karsiyaka, and O. F. Maden, "Normalized FxLMS algorithm focused active noise cancellation (ANC) system approach in automotive industry," in *2025 16th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, Türkiye, 2025, pp. 1–6. DOI: 10.1109/ELECO69582.2025.11329311

[5] Z. Ma and M. Lei, *Design of improved least mean square adaptive noise canceller*, Civil Aviation University of China.

[6] S. M. Kuo and D. R. Morgan, "Active noise control: A tutorial review," *Proceedings of the IEEE*, vol. 87, no. 6, pp. 943–973, 1999. DOI: 10.1109/5.763310

[7] D. Shi, C. Shi, and W.-S. Gan, "A systolic FxLMS structure for implementation of feedforward active noise control on FPGA," in *2016 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, Jeju, South Korea, 2016, pp. 1–6. DOI: 10.1109/APSIPA.2016.7820755

[8] T. Miyake, K. Iwai, and Y. Kajikawa, "Head-mounted multi-channel feedforward active noise control system for reducing noise arriving from various directions," *IEEE Access*, vol. 11, pp. 6935–6943, 2023. DOI: 10.1109/ACCESS.2023.3237812

[9] A. Klemd, B. Klauer, J. Timmermann, and D. Sachau, "A flexible multi-channel feedback FxLMS architecture for FPGA platforms," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, Dublin, Ireland, 2021, pp. 319–326. DOI: 10.1109/FPL53798.2021.00057

[10] C. Ji and W. Fan, "Hardware-accelerated real-time FIR filtering for audio applications using FPGAs," *Intelligent and Converged Networks*, vol. 6, no. 3, pp. 209–222, 2025. DOI: 10.23919/ICN.2025.0017

[11] Y. Pang, S. Ou, Z. Cai, and Y. Gao, "Posterior error energy minimization based combined FxNLMS and FxLMS algorithm for active noise control," *IEEE Access*, vol. 12, pp. 30 754–30 764, 2024. DOI: 10.1109/ACCESS.2024.3368860

[12] AMD, *Artix-7 FPGA data sheet: DC and AC switching characteristics (DS181)*, AMD/Xilinx, 2024. [Online]. Available: https://docs.amd.com/r/en-US/ds181-Artix-7-Data-Sheet

[13] Digilent, *Arty A7 reference manual*, Digilent Inc., 2025. [Online]. Available: https://digilent.com/reference/programmable-logic/arty-a7/reference-manual

[14] D. Pinardi et al., “An innovative architecture of full-digital microphone arrays over A2B network for consumer electronics,” *IEEE Transactions on Consumer Electronics*, vol. 68, no. 3, pp. 200–208, 2022. DOI: 10.1109/TCE.2022.3187453

[15] InvenSense, *INMP441 omnidirectional microphone with I2S digital output: Product specification*, InvenSense/TDK, 2016. [Online]. Available: https://invensense.tdk.com/products/inmp441/

[16] Texas Instruments, *PCM5102A 2VRMS DirectPath stereo audio DAC with 112/106-dB SNR and negative charge pump: Datasheet*, Texas Instruments, 2015. [Online]. Available: https://www.ti.com/product/PCM5102A